Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0126 | G02

INGENIERÍA DEL SOFTWARE

Laboratorio 2B

Introducción a Javascript

Profesora:

Msc. Rebeca Obando Vásquez

Javier Cruz Ruiz C02517

23 de Marzo, 2026

1. ¿Cuáles son los data types que soporta javascript ?

R/ Javascript soporta los siguientes tipos de datos:

- Null: contiene un valor nulo.
- Undefined: Indica la ausencia de un valor.
- Boolean: Es una entidad lógica, puede ser *true* o *false.*
- Number: Es un valor en formato binario de doble precisión de 64 bits. Permite almacenar números de punto flotante positivos, así como negativos.
- BigInt: Representa enteros de una magnitud arbitraria. Permite almacenar números más grandes que con el tipo Number.
- String: Representa texto. Cada elemento en el string representa una posición en la cadena, empezando por 0, siguiendo con el 1 y así sucesivamente.
- Symbol: Es una primitiva única que puede ser utilizada como llave de un Object. Crean propiedades únicas que no interfieren con otras llaves del código.

2. ¿Cómo se puede crear un objeto en javascript? De un ejemplo

R/ Para crear un objeto en Javascript, se puede usar un object initializer, o bien, crear un constructor e invocar el constructor con el operador *new.*

Ejemplo con object initializer:

```
const auto = {
  color: "gris",
  ruedas: 4,
  motor: { cilindros: 4, tamano: 1.6 },
};
```

Ejemplo con constructor:

```
function Auto(marca, modelo, año) {
  this.marca = marca;
  this.modelo = modelo;
  this.año = año;
```

```
}
```

```
const miAuto = new Auto("Suzuki", "Vitara", 2017);
```

3. ¿Cuáles son los alcances (scope) de las variables en javascript?

R/ Javascript tiene 3 tipos de alcance o scope:

- Global: Se declaran fuera de cualquier bloque o función. Se pueden accesar desde cualquier parte en un programa Javascript.
- De función: Las variables definidas dentro de una función no son accesibles o visibles fuera de esa función.
- De bloque: Las variables declaradas con *let* y *const* dentro de un bloque de código son accesibles sólo dentro de ese bloque (dentro de {}).

4. ¿Cuál es la diferencia entre undefined y null?

R/ La diferencia principal es que en undefined se crea una variable pero no se le asigna un valor, mientras que en null se asigna un valor nulo intencionalmente. Por ejemplo: let edad; -> undefined. let edad = null; -> null.

5. ¿Qué es el DOM?

R/ Significa Document Object Model. Es un modelo de objetos para documentos en HTML. Cuando se carga una página web, se crea una representación en forma de árbol, donde cada parte del documento HTML son nodos del árbol.

Con el DOM, podemos acceder a elementos HTML, la forma más común de hacerlo es utilizando el *id* del elemento. Un ejemplo de un método DOM es *getElementById(),* que retorna el elemento con el ID especificado en parámetros.

6. Usando Javascript se puede acceder a diferentes elementos del DOM. ¿Qué hacen, que retorna y para qué funcionan las funciones getElement y

querySelector? Cree un ejemplo

R/ Las funciones getElement incluyen opciones como getElementById(), getElementsByClassName(), getElementsByTagName(). Estas funciones nos ayudan a encontrar elementos HTML en el DOM dependiendo de la información que tengamos. Por ejemplo, en el caso de getElementById(), pasamos como parámetro el ID del elemento y nos retornaría el elemento especificado por el ID, esto resulta muy útil a la hora de necesitar comparar o modificar la información de dicho elemento (funciona de igual manera con getElementsByClassName() especificando la clase del elemento y getElementsByTagName() especificando el tag).

La función queryselector() retorna el primer elemento del documento que coincida con el grupo especificado de selectores. Por ejemplo, en el código var el = document.querySelector(".miClase"); se obtendrá el primer elemento del documento con la clase “miClase”.

7. Investigue cómo se pueden crear nuevos elementos en el DOM usando Javascript. De un ejemplo

R/ En Javascript, se pueden crear nuevos elementos HTML con el método document.createElement(). Un ejemplo para crear una nueva *div* es el siguiente:

```
const newDiv = document.createElement("div");
```

A esta nueva div que fue creada se le puede agregar contenido de la siguiente forma:

```
const newContent = document.createTextNode("Hi there and
greetings!");
newDiv.appendChild(newContent);
```

8. ¿Cuál es el propósito del operador this?

R/ *This* hace referencia al objeto que está ejecutando una función. Por ejemplo:

```
const persona = {
  nombre: "Ana",
  saludar: function() {
    console.log(this.nombre);
  }
};
```

This se refiere al nombre de la persona creada.

9. ¿Qué es un promise en Javascript?De un ejemplo

R/ Un objeto *Promise* representa el eventual éxito o fracaso de una operación asíncrona. Un promise se puede encontrar en uno de los siguientes estados:

- Pending: Estado inicial, ni completado ni rechazado.
- Fulfilled: La operación se completó correctamente.
- Rejected: La operación falló.

Ejemplo:

```
const promesa = new Promise((resolve, reject) => {
  let exito = true;
  if (exito) resolve("Todo bien");
  else reject("Error");
});

promesa.then(res => console.log(res))
       .catch(err => console.log(err));
```

10. ¿Qué es Fetch en Javascript? De un ejemplo

R/ Permite a los navegadores web hacer peticiones HTTP a los servidores web.

Ejemplo de fetch:

```
async function getData() {
  const url = "https://example.org/products.json";
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`Response status: ${response.status}`);
```

```
    }

    const result = await response.json();
    console.log(result);
  } catch (error) {
    console.error(error.message);
  }
}
```

Se declara un string que contiene la URL y luego se usa fetch(), pasando la URL.

11. ¿Qué es Asynch/Await en Javascript ? De un ejemplo

R/ Async y Await permiten trabajar con funciones asíncronas. Async indica que se devolverá una promesa. Await se usa dentro de funciones async para esperar el resultado de una promesa sin bloquear el resto del programa.

Ejemplo:

```
function resolveAfter2Seconds() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("resolved");
    }, 2000);
  });
}

async function asyncCall() {
  console.log("calling");
  const result = await resolveAfter2Seconds();
  console.log(result);
  // Expected output: "resolved"
}

asyncCall();
```

12. ¿Qué es un Callback? De un ejemplo

R/ Un callback es una función que se pasa como argumento a otra función, la cual se ejecuta más tarde para completar una tarea. Hay dos tipos:

- Callback asíncrono: Se ejecutan tiempo después, permitiendo que el programa principal corra sin esperar. Esto es importante ya que previene que la aplicación se congele durante tareas más largas.
- Callback sincrónico: Se ejecutan inmediatamente dentro de la otra función, bloqueando las demás operaciones hasta completarse.

Ejemplo:

```
function saludar(nombre, callback) {
    console.log("Hola " + nombre);
    callback();
}

function despedirse() {
    console.log("Adiós!");
}

saludar("Carlos", despedirse);
```

13. ¿Qué es Clousure?

R/ Previamente se definió que las variables pueden tener diferente scope. El closure se crea cuando una función recuerda las variables de fuera de su scope, incluso luego de que la función de fuera termina de ejecutarse.

Ejemplo:

```
function crearContador() {
    let contador = 0;

    return function() {
        contador++;
        return contador;
```

```
    };
}

const contar = crearContador();

console.log(contar()); // 1
console.log(contar()); // 2
console.log(contar()); // 3
```

14. ¿Cómo se puede crear un cookie usando Javascript?

R/ Las cookies son datos guardados en pequeños archivos de texto. Sin las cookies, al cerrar la conexión con un servidor web, se perdería la información ingresada inicialmente. Gracias a las cookies una página web puede "recordar" la información de un usuario.

Las cookies se almacenan en pares de nombre-valor, por ejemplo: username = John Doe.

En Javascript, se pueden crear, leer y borrar cookies con la propiedad *document.cookie.*

Un ejemplo de cómo crearlas es el siguiente:

```
document.cookie = "username=John Doe";
```

También se permite agregar una fecha de expiración, así como indicar a qué *path* pertenece la cookie:

```
document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013
12:00:00 UTC; path=/";
```

Leer una cookie se puede hacer de la siguiente forma, esto retornará todas las cookies en un sólo string.

```
let x = document.cookie;
```

Por último, para borrar una cookie, se puede editar y establecer la fecha de expiración a una fecha pasada:

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00
UTC; path=/;";
```

15. ¿Cuál es la diferencia entre var, let y const?

R/ La diferencia principal entre las 3 es que **var** declara variables con scope de función o globales y permite re-declarar y actualizarlas dentro del mismo scope, **let** declara variables en el scope de bloque permitiendo actualizarlas pero no re-declararlas dentro del mismo bloque, y **const** permite declarar variables con scope de bloque que no pueden ser reasignadas luego de su asignación inicial.

Ejemplo:

```
// var example
var x = 10;
var x = 20;   // Re-declaration allowed
x = 30;       // Update allowed
console.log(x); // Output: 30


// let example
let y = 10;
// let y = 20; // Re-declaration NOT allowed
y = 25;       // Update allowed
console.log(y); // Output: 25


// const example
const z = 10;
// z = 20;     // Re-assignment NOT allowed
console.log(z); // Output: 10
```

Enlace del repositorio de Github: https://github.com/ibra0610/my_labs

Referencias:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects
https://www.w3schools.com/js/js_scope.asp
https://www.w3schools.com/JS/js_htmldom.asp
https://www.w3schools.com/js/js_htmldom_elements.asp
https://developer.mozilla.org/es/docs/Web/API/Document/querySelector
https://developer.mozilla.org/en-US/docs/Web/API/Document/createElement
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
https://www.w3schools.com/js/js_api_fetch.asp
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function
https://www.w3schools.com/js/js_callback.asp
https://www.w3schools.com/js/js_function_closures.asp
https://www.w3schools.com/js/js_cookies.asp
https://www.geeksforgeeks.org/javascript/difference-between-var-let-and-const-keywords-in-javascript/